

# Esame FISM 2026 — Tema 4: Menu di un ristorante

## Esame FISM 2026 — Tema 4: Menu di un ristorante

### Scenario

Realizzare il sito multipagina di un ristorante fittizio con menu, categorie, prezzi e allergeni.

Il progetto deve essere un sito web multipagina realizzato con HTML, CSS, Bootstrap e dati caricati da un file `data.json` locale. Il file `script.js` fornito in questa cartella contiene già la logica base per leggere `data.json` e generare card e tabella.

### Obiettivo

Creare una repository GitHub ordinata e un sito navigabile che presenti i dati del tema assegnato in modo chiaro, leggibile e coerente.

### Mini guida: aprire il progetto

Questo tema usa `fetch("data.json")` per leggere i dati locali. Per questo motivo il sito deve essere aperto tramite un server locale: non basta fare doppio click su `index.html`.

Metodo consigliato da terminale, dentro la cartella del progetto:

```
python -m http.server 8000
```

Poi aprire nel browser:

```
http://localhost:8000
```

In alternativa puoi usare Live Server di Visual Studio Code.

### File forniti

In questa cartella trovi:

```
04-menu-di-un-ristorante/  
├── testo_esercizio.md  
├── script.js  
└── data.json
```

Dovrai creare tu il resto della repository del progetto.

### Struttura minima richiesta della repository

```
nome-progetto/  
├── README.md  
├── index.html  
├── menu.html  
├── prenotazioni.html  
├── style.css  
├── script.js  
├── data.json  
├── docs/  
│   ├── installazione.md  
│   ├── faq.md  
│   └── api.md  
└── assets/  
    └── immagini/
```

Il file `LICENSE` è facoltativo e vale come elemento bonus solo se la licenza scelta viene spiegata nel README.

## Pagine richieste

Realizzare almeno queste pagine:

- `index.html`
- `menu.html`
- `prenotazioni.html`

Ogni pagina deve avere:

- navbar Bootstrap funzionante;
- contenuto principale ordinato;
- footer;
- collegamenti corretti alle altre pagine;
- stile coerente con il tema.

## API locale da usare

In questo esercizio l'API è simulata dal file locale:

```
data.json
```

Il file `data.json` contiene piatti con prezzo, categoria, allergeni e disponibilità.

Il file `script.js` carica i dati con:

```
fetch("data.json")
```

Quindi il progetto deve essere aperto tramite un piccolo server locale, non solo con doppio click sul file HTML.

## Avvio locale consigliato

Aprire il terminale nella cartella del progetto ed eseguire:

```
python -m http.server 8000
```

Poi aprire nel browser:

```
http://localhost:8000
```

In alternativa è possibile usare l'estensione Live Server di Visual Studio Code.

## Elementi HTML richiesti per usare lo script

Nella pagina in cui vuoi mostrare i dati, ad esempio `menu.html`, inserire questi elementi:

```

<section class="container my-5">
  <h2 class="mb-4">Dati del progetto</h2>

  <div id="status-message"></div>

  <div class="row g-3 mb-4">
    <div class="col-md-8">
      <input id="search-input" class="form-control" type="search" placeholder="Cerca...">
    </div>
    <div class="col-md-4">
      <select id="category-filter" class="form-select"></select>
    </div>
  </div>

  <p class="text-body-secondary">
    Elementi mostrati: <span id="items-count">0</span>
  </p>

  <div id="cards-container" class="row g-4"></div>
</section>

```

Per mostrare anche una tabella, inserire in una pagina o sezione:

```

<section class="container my-5">
  <h2 class="mb-4">Tabella dati</h2>
  <div id="table-container"></div>
</section>

```

Collegare lo script prima della chiusura di `</body>`:

```
<script src="script.js"></script>
```

## Campi principali del dataset

Il file `data.json` contiene oggetti con questi campi principali:

```

id
title
subtitle
category
description
fields

```

Nel sito devi mostrare almeno:

- `title`: Piatto
- `category`: Categoria
- `fields.prezzo`: Prezzo
- `fields.allergeni`: Allergeni
- `fields.disponibilita`: Disponibilità

## Requisiti tecnici

Il sito deve contenere:

- almeno 3 pagine HTML;
- Bootstrap collegato correttamente;
- navbar responsive Bootstrap;
- almeno una griglia con `container`, `row` e `col`;
- card Bootstrap generate dai dati;

- almeno una tabella Bootstrap o tabella responsive;
- file `style.css` con personalizzazione reale;
- uso di immagini salvate in `assets/immagini/` con percorsi relativi;
- messaggio di errore leggibile se `data.json` non viene caricato.

## Documentazione richiesta

### README.md

Il README deve contenere:

- titolo del progetto;
- breve descrizione;
- funzionalità principali;
- tecnologie usate;
- istruzioni rapide per aprire il sito;
- requisiti necessari per aprire ed eseguire il progetto;
- struttura della repository;
- link ai documenti nella cartella `docs/`;
- autore;
- eventuale licenza scelta.

### docs/installazione.md

Il file deve spiegare:

- come clonare la repository;
- come avviare il server locale;
- quale pagina aprire nel browser;
- eventuali problemi legati a `fetch("data.json")`.

### docs/faq.md

Il file deve contenere almeno 5 domande frequenti con risposta.

Esempi di domande possibili:

- Il sito funziona senza internet?
- Perché i dati non compaiono?
- Come modifico i contenuti?
- Dove si trova il file dei dati?
- A cosa serve il file `script.js`?

### docs/api.md

Il file deve spiegare:

- che in questo progetto l'API è simulata da `data.json`;
- quali dati contiene;
- quali campi vengono usati;
- dove vengono mostrati nel sito;
- come viene caricato il file tramite JavaScript.

## Bonus possibili

- +5 se il sito è bello, curato e coerente graficamente.
- +5 se GitHub Pages funziona ed è linkato nel README.
- +5 se è presente una licenza e viene spiegato perché è stata scelta.
- +5 se la documentazione contiene screenshot e/o GIF utili.

## Checklist finale

Prima della consegna controllare:

- ☐ La repository contiene tutti i file richiesti.
- ☐ La navbar collega correttamente tutte le pagine.
- ☐ Bootstrap è caricato.
- ☐ `style.css` è collegato.
- ☐ `script.js` è collegato.
- ☐ `data.json` viene caricato tramite server locale.
- ☐ Le card vengono generate correttamente.
- ☐ La tabella viene mostrata correttamente, se usata.
- ☐ Le immagini usano percorsi relativi.
- ☐ Il README contiene i link alla documentazione.
- ☐ `docs/installazione.md`, `docs/faq.md` e `docs/api.md` sono presenti e leggibili.